REPUBLIQUE DU CAMEROUN

PAIX – TRAVAIL – PATRIE

MINISTERE DE L'ENSEIGNEMENT SUPERIEURE

REPUBLIC OF CAMEROON

PEACE – WORK – FATHERLAND

MINISTRY OF HIGHER EDUCATION

FACULTY OF ENGINEERING AND TECHNOLOGY

PO BOX 63

BUEA, SOUTH WEST REGION

DEPARTMENT OF COMPUTER ENGINEERING

COURSE CODE: CEF440

COURSE TITLE: INTERNET PROGRAMMING AND MOBILE DEVELOPMENT

COURSE INSTRUCTOR: Dr. NKEMENI VALERY

TASK 6 REPORT: Database Design and Backend Implementation

Presented by:

NAMES	MATRICLE
1. **MEKOLLE ASHLEY ARREYMANYOR**	**FE22A247**
2. **AKO RUTH ACHERE**	**FE22A144**
3. **NGUIMENANG ZEUFACK KEREINE**	**FE22A266**
4. **TUMUTOH LYDIE-KASEY BIHNUI**	**FE22A321**
5. **TIWA DELAN NDIKA**	**FE22A319**

JUNE 2025

Table of Contents

1. Introduction

1.1. Purpose

The database is the **central information hub** for SmartCheck, governing everything from user authentication data to fine-grained attendance logs. It must accurately persist:

- **User profiles** (students, lecturers, admins)
- **Course definitions and enrolments**
- **Attendance sessions** (time window, geofence, lecturer in charge)
- **Biometric check-in records** (face-match metadata, GPS coordinates)
- **Disputes and leave requests**
  By storing this data in an organized, query-friendly structure, the database enables SmartCheck to deliver its core promise: **instant, verifiable attendance** without manual roll-calls or paper sheets.

1.2. Importance of a Well-Structured & Scalable Design

Because SmartCheck is expected to serve multiple faculties and potentially thousands of students per semester, the data layer must be both **predictable** and **horizontally scalable**. A poorly designed schema would lead to:

- Latency spikes during mass check-ins
- Synchronization conflicts between parallel sessions
- Complicated maintenance when new features (e.g., analytics, push notifications) are added
  A thoughtfully modelled database avoids these pitfalls by clarifying entity relationships (users → courses → sessions → attendance) and by supporting fast, indexed look-ups for real-time screens such as dashboards and lecture-hall monitors.

1.3. Chosen Technology: Firebase Firestore

SmartCheck uses **Firebase Firestore**, Google's cloud-hosted NoSQL document database. Firestore was selected because it offers:

- **Real-time synchronization**: attendance logs update instantly on every lecturer and student device.
- **Automatic scaling**: collection sharding keeps read/write latency low even during campus-wide exams.

- **Granular security rules**: access can be limited per document (e.g., a student may read only their own disputes).
- **Native integration with Firebase Authentication**: token claims (role: student | lecturer | admin) flow directly into Firestore rules without extra middleware.
- **Serverless operation**: no manual cluster management; ideal for academic deployments on limited DevOps budgets.

1.4. Database Support for SmartCheck's Key Workflows

A robust Firestore schema underpins SmartCheck's two signature technologies:

1. **Facial Recognition workflow**
    - Stores reference face embeddings or image URLs under each user profile.
    - Logs confidence scores and timestamps with each check-in document for audit trails.
2. **Geofencing workflow**
    - Saves latitude, longitude, and radius for every active session.
    - Compares incoming GPS coordinates inside Cloud Functions before writing an attendance record.

Because both workflows are mobile-first, Firestore's offline persistence and batched writes guarantee that check-ins remain dependable even on unstable campus Wi-Fi. In sum, the database layer is not merely a data dump; it is an **active enforcement engine** that validates presence, preserves integrity, and feeds upstream features such as analytics and push notifications.

2. Data Elements

SmartCheck's data layer revolves around six primary entities. Each entity is designed to be atomic, query-friendly, and easily extensible as new features (e.g., additional biometrics or analytics) are introduced.

2.1. User

Represents every authenticated person in the system—students, lecturers, and admins. *Core attributes* include personal info (name, email, phone), institutional metadata (role, department), face-template reference, and FCM push-token. *Why it matters*: establishes identity and role-based access for all downstream modules.

2.1.1. Common User Data

Field Name	Data Type	Constraints	Description	Example Values
userID	UUID	Primary Key, Not Null, Auto-generated	Unique identifier for each user	"550e8400-e29b-41d4-a716-446655440000"
passwordHash	VARCHAR(255)	Not Null, BCrypt encrypted	Encrypted password using BCrypt algorithm	"$2b$10$N9qo8..."
firstName	VARCHAR(100)	Not Null, 1-100 characters	User's first name, alphabetic characters only	"John", "Mary"
lastName	VARCHAR(100)	Not Null, 1-100 characters	User's last name, alphabetic characters only	"Doe", "Smith"
email	VARCHAR(255)	Unique, Not Null, Valid email format	User's email address for communication	"john.doe@university.edu"
role	ENUM	Not Null, Values: 'Student', 'Lecturer', 'Admin'	User role determining system access level	"Student", "Lecturer", "Admin"
phoneNumber	VARCHAR(20)	Optional, International format	Contact number with country code	"+237123456789", "+1-555-0123"
registrationDate	TIMESTAMP	Not Null, Auto-generated	Date and time when user registered	"2024-01-15 14:30:25"
profileImageURL	VARCHAR(500)	Optional	URL to user's profile image	"https://storage.../profile123.jpg"
passwordResetToken	VARCHAR(255)	Nullable	Token for password reset functionality	"abc123def456..."
tokenExpiryDate	TIMESTAMP	Nullable	Expiration date for reset token	"2024-06-06 12:00:00"

2.1.2. Student-Specific Data

Field Name	Data Type	Constraints	Description	Example Values
studentID	VARCHAR(20)	Primary Key, Not Null, Unique	Institution-assigned student identifier	"STU2024001", "20240001"
enrollmentNumber	VARCHAR(30)	Unique, Not Null	Official enrollment/matriculation number	"EN/2024/CSE/001"
department	VARCHAR(100)	Not Null	Student's academic department	"Computer Science", "Mechanical Engineering"
semester	INTEGER	Not Null, Range: 1-8	Current semester number	1, 2, 3, 4, 5, 6, 7, 8
yearOfStudy	INTEGER	Not Null, Range: 1-4	Academic year level	1, 2, 3, 4
program	VARCHAR(100)	Not Null	Degree program enrolled in	"Bachelor of Science", "Master of Engineering"
admissionYear	INTEGER	Not Null	Year of admission to institution	2024, 2023, 2022
emergencyContact	VARCHAR(20)	Optional	Emergency contact number	"+237555123456"

2.1.3. Lecturer-Specific Data

Field Name	Data Type	Constraints	Description	Example Values
facultyID	VARCHAR(20)	Primary Key, Not Null, Unique	Unique faculty identifier	"FAC2024001", "LECT001"
department	VARCHAR(100)	Not Null	Lecturer's academic department	"Computer Science", "Mathematics"
designation	VARCHAR(100)	Not Null	Academic position/title	"Assistant Professor", "Senior Lecturer", "Professor"
specialization	VARCHAR(200)	Optional	Area of expertise/specialization	"Machine Learning", "Database Systems"

Field Name	Data Type	Constraints	Description	Example Values
officeLocation	VARCHAR(100)	Optional	Physical office location	"Block A, Room 201", "Engineering Building 3F"
officeHours	VARCHAR(200)	Optional	Available consultation hours	"Mon-Wed 2:00-4:00 PM", "Tue/Thu 10:00-12:00"
qualifications	TEXT	Optional	Educational qualifications	"PhD Computer Science, MSc Software Engineering"
joinDate	DATE	Not Null	Date of joining institution	"2020-08-15"

2.1.4. Admin-Specific Data

Field Name	Data Type	Constraints	Description	Example Values
adminID	VARCHAR(20)	Primary Key, Not Null, Unique	Unique admin identifier	"ADM2024001", "ADMIN001"
permissions	JSON	Not Null	Specific admin permissions array	["USER_MANAGEMENT", "SYSTEM_CONFIG"]
department	VARCHAR(100)	Optional	Admin's associated department	"IT Department", "Academic Affairs"
lastActivityDate	TIMESTAMP	Auto-updated	Last admin activity timestamp	"2024-06-05 15:30:00"

2.2. Course

Defines an academic course or class section. It stores the course code, title, semester, owning lecturer ID, and the list of enrolled student IDs.
Why it matters: serves as the anchor for grouping sessions, attendance records, and analytics.

2.2.1. Course Data

Field Name	Data Type	Constraints	Description	Example Values
courseID	UUID	Primary Key, Not Null, Auto-generated	Unique course identifier	"550e8400-e29b-41d4-a716-446655440001"
courseCode	VARCHAR(20)	Unique, Not Null, Uppercase	Institution course code	"CEF440", "CS101", "MATH205"
courseName	VARCHAR(200)	Not Null	Full descriptive course name	"Software Engineering", "Introduction to Programming"
courseDescription	TEXT	Optional	Detailed course description	"This course covers fundamental principles..."
credits	INTEGER	Not Null, Range: 1-6	Course credit hours	3, 4, 2, 6
semester	INTEGER	Not Null, Range: 1-8	Course semester	1, 2, 3, 4, 5, 6, 7, 8
academicYear	VARCHAR(9)	Not Null, Format: YYYY/YYYY	Academic year period	"2024/2025", "2023/2024"
lecturerID	VARCHAR(20)	Foreign Key, Not Null	Assigned lecturer identifier	"FAC2024001"
currentEnrollment	INTEGER	Not Null, Default: 0	Current number of enrolled students	45, 67, 123
enrollmentStatus	ENUM	Not Null, Values: 'OPEN', 'CLOSED', 'FULL'	Enrollment availability status	"OPEN", "CLOSED", "FULL"
courseType	ENUM	Not Null	Course delivery type	"LECTURE", "LAB", "HYBRID", "ONLINE"
schedule	JSON	Not Null	Weekly schedule information	{"Monday": "09:00-11:00", "Wednesday": "14:00-16:00"}
venue	VARCHAR(100)	Optional	Default classroom location	"Room 101, Engineering Block", "Lab 205"

Field Name	Data Type	Constraints	Description	Example Values
createdDate	TIMESTAMP	Not Null, Auto-generated	Course creation timestamp	"2024-01-10 10:30:00"
isActive	BOOLEAN	Not Null, Default: true	Course active status	true, false

2.2.2. Course Enrollment Data

Field Name	Data Type	Constraints	Description	Example Values
enrollmentID	UUID	Primary Key, Not Null, Auto-generated	Unique enrollment identifier	"550e8400-e29b-41d4-a716-446655440002"
studentID	VARCHAR(20)	Foreign Key, Not Null	Enrolled student identifier	"STU2024001"
courseID	UUID	Foreign Key, Not Null	Course identifier	"550e8400-e29b-41d4-a716-446655440001"
enrollmentDate	TIMESTAMP	Not Null, Auto-generated	Date and time of enrollment	"2024-01-15 09:30:00"
enrollmentStatus	ENUM	Not Null	Current enrollment status	"ACTIVE", "INACTIVE", "WITHDRAWN", "COMPLETED"
attendancePercentage	DECIMAL(5,2)	Default: 0.00	Overall attendance percentage	85.50, 92.75, 67.25
withdrawalDate	TIMESTAMP	Optional	Date of course withdrawal	"2024-03-15 14:20:00"
withdrawalReason	TEXT	Optional	Reason for withdrawal	"Medical reasons", "Schedule conflict"

2.2.3. Class Schedule Data

Field Name	Data Type	Constraints	Description	Example Values
scheduleID	UUID	Primary Key, Not Null, Auto-generated	Unique schedule identifier	"550e8400-e29b-41d4-a716-446655440011"
courseID	UUID	Foreign Key, Not Null	Associated course identifier	"550e8400-e29b-41d4-a716-446655440001"

Field Name	Data Type	Constraints	Description	Example Values
dayOfWeek	INTEGER	Not Null, Range: 1-7	Day of week (1=Monday, 7=Sunday)	1, 3, 5
startTime	TIME	Not Null	Scheduled start time	"09:00:00", "14:30:00"
endTime	TIME	Not Null	Scheduled end time	"11:00:00", "16:30:00"
geofenceID	UUID	Foreign Key, Not Null	Associated location geofence	"550e8400-e29b-41d4-a716-446655440004"
effectiveStartDate	DATE	Not Null	When schedule becomes active	"2024-01-15"
effectiveEndDate	DATE	Optional	When schedule expires	"2024-06-30"
sessionType	ENUM	Not Null	Type of scheduled session	"LECTURE", "LAB", "TUTORIAL", "EXAM"
createdBy	VARCHAR(20)	Foreign Key, Not Null	Lecturer who created schedule	"FAC2024001"
createdDate	TIMESTAMP	Not Null, Auto-generated	Schedule creation timestamp	"2024-01-10 08:00:00"

2.3. Session

A time-bound, geofenced attendance window created by a lecturer for a specific course. It records start/end timestamps, GPS centre point, radius, and session status (open, closed). *Why it matters*: enforces location and time constraints for valid check-ins.

2.3.1. Course Session Data

Field Name	Data Type	Constraints	Description	Example Values
sessionID	UUID	Primary Key, Not Null, Auto-generated	Unique session identifier	"550e8400-e29b-41d4-a716-446655440003"
courseID	UUID	Foreign Key, Not Null	Associated course identifier	"550e8400-e29b-41d4-a716-446655440001"

Field Name	Data Type	Constraints	Description	Example Values
sessionDate	DATE	Not Null	Date of session	"2024-06-05"
startTime	TIME	Not Null	Session start time	"09:00:00", "14:30:00"
endTime	TIME	Not Null	Session end time	"11:00:00", "16:30:00"
actualStartTime	TIMESTAMP	Optional	Actual session start timestamp	"2024-06-05 09:05:30"
actualEndTime	TIMESTAMP	Optional	Actual session end timestamp	"2024-06-05 10:58:15"
locationDescription	VARCHAR(200)	Not Null	Classroom/ venue description	"Room 101, Main Building", "Computer Lab 3"
sessionStatus	ENUM	Not Null	Current session status	"SCHEDULED", "OPEN", "CLOSED", "CANCELLED", "COMPLETED"
attendanceCount	INTEGER	Default: 0	Number of students who attended	45, 32, 67
expectedAttendance	INTEGER	Not Null	Expected number of attendees	50, 35, 70
createdBy	VARCHAR(20)	Foreign Key, Not Null	Lecturer who created session	"FAC2024001"
createdDate	TIMESTAMP	Not Null, Auto-generated	Session creation timestamp	"2024-06-01 08:30:00"
modifiedDate	TIMESTAMP	Auto-updated	Last modification timestamp	"2024-06-03 16:45:00"

2.4. Geofencing Data Elements

2.4.1. Geofence Location Data

Field Name	Data Type	Constraints	Description	Example Values
geofenceID	UUID	Primary Key, Not Null, Auto-generated	Unique geofence identifier	"550e8400-e29b-41d4-a716-446655440004"
centerLatitude	DECIMAL(10,8)	Not Null, Range: -90 to 90	Center point latitude coordinate	4.12345678, -23.87654321
centerLongitude	DECIMAL(11,8)	Not Null, Range: -180 to 180	Center point longitude coordinate	9.87654321, -120.12345678
radiusInMeters	INTEGER	Not Null, Range: 10-1000	Geofence radius in meters	50, 100, 200
venue	VARCHAR(200)	Not Null	Descriptive venue/classroom name	"Main Auditorium", "Computer Lab 1", "Library Study Hall"
isActive	BOOLEAN	Not Null, Default: true	Geofence operational status	true, false
createdDate	TIMESTAMP	Not Null, Auto-generated	Geofence creation timestamp	"2024-01-05 12:00:00"

2.5. Facial Recognition Data Elements

2.5.1. Facial Data

Field Name	Data Type	Constraints	Description	Example Values
facialDataID	UUID	Primary Key, Not Null, Auto-generated	Unique facial data identifier	"550e8400-e29b-41d4-a716-446655440005"
studentID	VARCHAR(20)	Foreign Key, Not Null, Unique	Associated student identifier	"STU2024001"
facialEmbedding	BLOB	Not Null, Encrypted	Biometric facial features/descriptors	[Encrypted binary data]
registrationDate	TIMESTAMP	Not Null, Auto-generated	Date facial data was registered	"2024-01-15 10:30:00"
isActive	BOOLEAN	Not Null, Default: true	Facial data operational status	true, false

Field Name	Data Type	Constraints	Description	Example Values
confidenceThreshold	DECIMAL(4,3)	Not Null, Range: 0.500-0.999	Minimum match confidence required	0.850, 0.900, 0.750
imageQualityScore	DECIMAL(4,3)	Optional, Range: 0.000-1.000	Quality score of registered images	0.925, 0.875
lastSuccessfulMatch	TIMESTAMP	Optional	Last successful face recognition	"2024-06-05 09:15:30"

2.6. Attendance

A single check-in record linked to a student and a session. Stores the captured GPS coordinates, face-match confidence, timestamp, and final status (Present, Late, Rejected). *Why it matters*: provides the immutable audit trail required for reporting and dispute resolution.

2.6.1. Attendance Record Data

Field Name	Data Type	Constraints	Description	Example Values
attendanceID	UUID	Primary Key, Not Null, Auto-generated	Unique attendance record identifier	"550e8400-e29b-41d4-a716-446655440006"
studentID	VARCHAR(20)	Foreign Key, Not Null	Student who checked in	"STU2024001"
sessionID	UUID	Foreign Key, Not Null	Associated session identifier	"550e8400-e29b-41d4-a716-446655440003"
courseID	UUID	Foreign Key, Not Null	Associated course identifier	"550e8400-e29b-41d4-a716-446655440001"
checkInTime	TIMESTAMP	Not Null, Auto-generated	Timestamp of check-in	"2024-06-05 09:05:30"
checkOutTime	TIMESTAMP	Optional	Timestamp of check-out	"2024-06-05 10:55:15"
attendanceStatus	ENUM	Not Null	Attendance status	"PRESENT", "ABSENT", "LATE", "EXCUSED", "PARTIAL"
verificationMethod	ENUM	Not Null	How attendance was verified	"SELF_CHECKIN", "LECTURER_ASSISTED", "MANUAL_ENTRY"

Field Name	Data Type	Constraints	Description	Example Values
locationLatitude	DECIMAL(10, 8)	Not Null	GPS latitude at check-in	4.12345678
locationLongitude	DECIMAL(11, 8)	Not Null	GPS longitude at check-in	9.87654321
locationAccuracy	DECIMAL(8,2)	Not Null	GPS accuracy in meters	3.50, 8.25, 15.75
distanceFromGeofence	DECIMAL(8,2)	Not Null	Distance from geofence center	15.25, 35.80, 2.10
faceMatchScore	DECIMAL(4,3)	Optional, Range: 0.000-1.000	Facial recognition confidence score	0.925, 0.875, 0.800
ipAddress	VARCHAR(45)	Not Null	IP address of check-in device	"192.168.1.100", "2001:0db8:85a3::8a2e:0370:7334"
verificationDate	TIMESTAMP	Optional	Date of manual verification	"2024-06-05 11:30:00"
isDisputed	BOOLEAN	Default: false	Whether attendance is under dispute	true, false
lastModified	TIMESTAMP	Auto-updated	Last record modification	"2024-06-05 16:45:00"

2.7. Dispute / Leave Request

Submitted by students to contest an attendance record or request authorised absence. Contains references to the affected session, reason text, optional evidence URL, and resolution status (pending, approved, denied).
Why it matters: formalises exception handling and keeps an auditable history of decisions.

2.7.1. Leave Request Data

Field Name	Data Type	Constraints	Description	Example Values
leaveRequestID	UUID	Primary Key, Not Null, Auto-generated	Unique leave request identifier	"550e8400-e29b-41d4-a716-446655440007"

Field Name	Data Type	Constraints	Description	Example Values
studentID	VARCHAR(20)	Foreign Key, Not Null	Student requesting leave	"STU2024001"
courseID	UUID	Foreign Key, Optional	Associated course identifier	"550e8400-e29b-41d4-a716-446655440001"
leaveType	ENUM	Not Null	Type of leave request	"MEDICAL", "PERSONAL", "EMERGENCY", "FAMILY", "ACADEMIC", "OTHER"
severity	ENUM	Default: 'NORMAL'	Leave request severity/priority	"LOW", "NORMAL", "HIGH", "CRITICAL"
startDate	DATE	Not Null	Leave start date	"2024-06-10"
endDate	DATE	Not Null	Leave end date	"2024-06-12"
totalDays	INTEGER	Computed, Range: 1-365	Total number of leave days	1, 3, 7, 14
reason	TEXT	Not Null, Max: 1000 chars	Detailed reason for leave	"Medical appointment with specialist"
supportingDocument	JSON	Optional	Attached files/documents	[{"filename": "medical_cert.pdf", "url": "..."}]
emergencyContact	VARCHAR(20)	Optional	Emergency contact during leave	"+237123456789"
requestDate	TIMESTAMP	Not Null, Auto-generated	Date request was submitted	"2024-06-08 14:30:00"
status	ENUM	Not Null, Default: 'PENDING'	Current request status	"PENDING", "APPROVED", "DENIED", "CANCELLED"
reviewedBy	VARCHAR(20)	Foreign Key, Optional	Lecturer/Admin who reviewed	"FAC2024001"
reviewDate	TIMESTAMP	Optional	Date of review	"2024-06-08 16:45:00"
reviewComments	TEXT	Optional	Reviewer's comments	"Approved due to medical emergency"
approvalDate	TIMESTAMP	Optional	Date of approval/denial	"2024-06-08 17:00:00"

2.7.2. Dispute Data

Field Name	Data Type	Constraints	Description	Example Values
disputeID	UUID	Primary Key, Not Null, Auto-generated	Unique dispute identifier	"550e8400-e29b-41d4-a716-446655440008"
studentID	VARCHAR(20)	Foreign Key, Not Null	Student raising dispute	"STU2024001"
sessionID	UUID	Foreign Key, Optional	Associated session identifier	"550e8400-e29b-41d4-a716-446655440003"
courseID	UUID	Foreign Key, Not Null	Associated course identifier	"550e8400-e29b-41d4-a716-446655440001"
attendanceRecordID	UUID	Foreign Key, Optional	Related attendance record	"550e8400-e29b-41d4-a716-446655440006"
disputeType	ENUM	Not Null	Type of dispute	"ATTENDANCE_ERROR", "TECHNICAL_ISSUE", "GEOFENCE_ERROR", "FACE_RECOGNITION_FAIL", "SYSTEM_MALFUNCTION", "OTHER"
disputeCategory	ENUM	Not Null	Dispute category	"TECHNICAL", "PROCEDURAL", "ADMINISTRATIVE", "APPEAL"
severity	ENUM	Default: 'MEDIUM'	Dispute severity level	"LOW", "MEDIUM", "HIGH", "CRITICAL"
description	TEXT	Not Null, Max: 2000 chars	Detailed description	"I was present in class but face recognition..."
evidenceFile	JSON	Optional	Supporting evidence/screenshots	[{"filename": "evidence.jpg", "url": "..."}]

Field Name	Data Type	Constraints	Description	Example Values
locationDuringIncident	JSON	Optional	Location during the incident	{"lat": 4.12345, "lng": 9.87654}
submissionDate	TIMESTAMP	Not Null, Auto-generated	Date dispute was submitted	"2024-06-05 15:30:00"
status	ENUM	Not Null, Default: 'PENDING'	Current dispute status	"PENDING", "UNDER_REVIEW", "RESOLVED", "REJECTED", "ESCALATED"
assignedTo	VARCHAR(20)	Foreign Key, Optional	Lecturer/Admin assigned to resolve	"FAC2024001"
assignmentDate	TIMESTAMP	Optional	Date dispute was assigned	"2024-06-05 16:00:00"
ResolutionDate	TIMESTAMP	Optional	Actual resolution timestamp	"2024-06-07 14:30:00"
resolutionTime	INTEGER	Computed	Resolution time in hours	23, 48, 72
resolutionComments	TEXT	Optional	Detailed resolution explanation	"Attendance marked present after verification"

3. Conceptual Design

The conceptual design defines the high-level data model for the SmartCheck system. It focuses on the **core entities**, their **relationships**, and how these reflect the operational logic of the application. Since the system is implemented using **Firebase Firestore (a NoSQL document-oriented database)**, the design approach differs from traditional relational models.

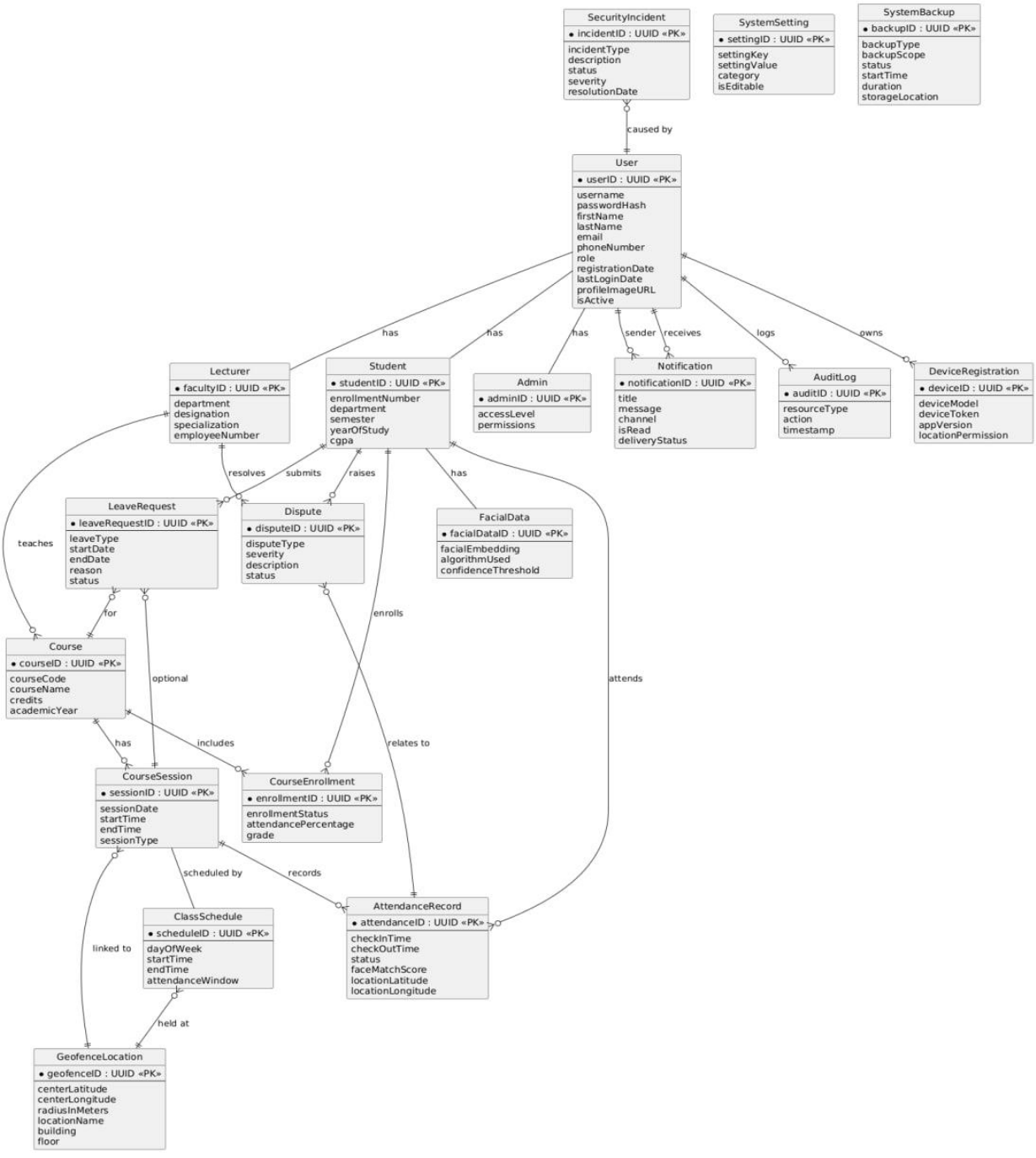
3.1. Overview of the Data Model Structure

SmartCheck uses a **collection-document** model, where each major entity (e.g., user, course, attendance session) is a top-level collection containing structured documents. Each document is self-contained, containing all relevant fields and references needed for that entity.

Instead of strict table relationships and joins (as in SQL), SmartCheck uses **document references**, **nested fields**, and **indexed queries** to model relationships. This approach supports high-speed reads and writes, real-time syncing, and offline-first access, which is essential for mobile-based systems.

3.2. Entity-Relationship (ER) Diagram

The ER Diagram serves as the blueprint for the database, visually representing the logical structure of the data. It helps in understanding the different pieces of information the system manages and how they interrelate.



Entity-Relationship Diagram for the SmartCheck System

3.3. Major Entities in the ER Diagram

The SmartCheck system revolves around several core entities, each representing a distinct real-world object or concept:

- **User:** The central entity representing any individual interacting with the system.
    - **Student, Lecturer, Admin (Specializations of User):** Inherit from User and add role-specific attributes.
- **Course:** Represents academic courses.
- **CourseEnrollment:** Manages the many-to-many relationship between Students and Courses.
- **CourseSession:** Represents a specific instance of a class or lecture.
- **ClassSchedule:** Defines recurring patterns for course sessions.
- **GeofenceLocation:** Stores geographical zone definitions.
- **FacialData:** Contains biometric facial embeddings for students.
- **AttendanceRecord:** Logs student attendance for sessions.
- **LeaveRequest:** Manages student requests for absence.
- **Dispute:** Handles student-raised concerns or discrepancies.
- **Notification:** Manages system-generated alerts.
- **SystemSetting:** Stores configurable application parameters.
- **AuditLog:** Records user and system actions.
- **DeviceRegistration:** Manages mobile devices registered by users.
- **SecurityIncident:** Logs detected security-related events.
- **SystemBackup:** Tracks system backup information

3.4. Relationships Between Major Entities

- **One-to-Many (Lecturer → Courses):** A lecturer can create and manage multiple courses. Each course document stores the lecturer's user ID to establish ownership.

- **One-to-Many (Course → Sessions):** Each course can have many attendance sessions. Sessions are stored in a separate collection with a reference to the courseId.
- **One-to-Many (Session → Attendance Logs):** For each session, multiple students will check in. Each attendance record logs the sessionId and the student's userId.
- **Many-to-One (Attendance → User & Session):** Every attendance log references both the student and the session in which it occurred.
- **One-to-Many (Student → Disputes):** Students can raise multiple disputes tied to different sessions. Each dispute links to a session and the student.
- **One-to-Many (User → Devices):** Each user may have one or more registered devices with FCM tokens for push notifications.

This structure promotes flexibility and denormalization, ensuring that documents can be fetched quickly without requiring complex server-side joins.

4. Relational Database Schema

The ER Diagram is translated into a set of relational tables. The following SQL Data Definition Language (DDL) statements define the complete schema for the SmartCheck database, including tables, columns, data types, primary keys, foreign keys, unique constraints, not null constraints, default values, and check constraints as derived from the "Data Elements" document.

4.1. User Management Tables

```
CREATE TABLE Users (
    userID UUID PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL CHECK (LENGTH(username) >=
3),
    passwordHash VARCHAR(255) NOT NULL,
    firstName VARCHAR(100) NOT NULL,
    lastName VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL, -- Consider adding CHECK for
email format
    role ENUM('Student', 'Lecturer', 'Admin') NOT NULL,
    phoneNumber VARCHAR(20),
    registrationDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    lastLoginDate TIMESTAMP,
    isActive BOOLEAN NOT NULL DEFAULT TRUE,
    profileImageURL VARCHAR(500),
    twoFactorEnabled BOOLEAN NOT NULL DEFAULT FALSE,
    accountLockoutCount INTEGER NOT NULL DEFAULT 0,
    passwordResetToken VARCHAR(255),
    tokenExpiryDate TIMESTAMP
);

CREATE TABLE Students (
    studentID VARCHAR(20) PRIMARY KEY,
    enrollmentNumber VARCHAR(30) UNIQUE NOT NULL,
    department VARCHAR(100) NOT NULL,
    semester INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 8),
    yearOfStudy INTEGER NOT NULL CHECK (yearOfStudy BETWEEN 1 AND 4),
    program VARCHAR(100) NOT NULL,
    admissionYear INTEGER NOT NULL,
    cgpa DECIMAL(3,2) CHECK (cgpa IS NULL OR cgpa BETWEEN 0.00 AND
4.00),
    guardianName VARCHAR(200),
    guardianPhone VARCHAR(20),
    emergencyContact VARCHAR(20),
    FOREIGN KEY (studentID) REFERENCES Users(userID) ON DELETE CASCADE
);

CREATE TABLE Lecturers (
    facultyID VARCHAR(20) PRIMARY KEY,
    department VARCHAR(100) NOT NULL,
    employeeNumber VARCHAR(30) UNIQUE NOT NULL,
    designation VARCHAR(100) NOT NULL,
    specialization VARCHAR(200),
    officeLocation VARCHAR(100),
    officeHours VARCHAR(200),
    qualifications TEXT,
    joinDate DATE NOT NULL,
    canCreateCourses BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (facultyID) REFERENCES Users(userID) ON DELETE CASCADE
);
CREATE TABLE Admins (
```

```
    adminID VARCHAR(20) PRIMARY KEY,
    accessLevel INTEGER NOT NULL CHECK (accessLevel BETWEEN 1 AND 5),
    permissions JSON NOT NULL,
    department VARCHAR(100),
    canManageUsers BOOLEAN NOT NULL DEFAULT FALSE,
    canViewReports BOOLEAN NOT NULL DEFAULT TRUE,
    canModifySettings BOOLEAN NOT NULL DEFAULT FALSE,
    lastActivityDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (adminID) REFERENCES Users(userID) ON DELETE CASCADE
);
```

4.2. Course and Session Management Tables

```
CREATE TABLE Courses (
    courseID UUID PRIMARY KEY,
    courseCode VARCHAR(20) UNIQUE NOT NULL,
    courseName VARCHAR(200) NOT NULL,
    courseDescription TEXT,
    credits INTEGER NOT NULL CHECK (credits BETWEEN 1 AND 6),
    semester INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 8),
    academicYear VARCHAR(9) NOT NULL, -- Consider CHECK for YYYY/YYYY
format
    lecturerID VARCHAR(20) NOT NULL,
    maxStudents INTEGER NOT NULL CHECK (maxStudents BETWEEN 1 AND
500),
    currentEnrollment INTEGER NOT NULL DEFAULT 0,
    enrollmentStatus ENUM('OPEN', 'CLOSED', 'FULL') NOT NULL,
    courseType ENUM('LECTURE', 'LAB', 'HYBRID', 'ONLINE', 'TUTORIAL',
'SEMINAR', 'EXAM') NOT NULL,
    prerequisites JSON,
    schedule JSON NOT NULL,
    location VARCHAR(100),
    createdDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    isActive BOOLEAN NOT NULL DEFAULT TRUE,
    FOREIGN KEY (lecturerID) REFERENCES Lecturers(facultyID) ON DELETE
RESTRICT
);



CREATE TABLE CourseEnrollments (
```

```
    enrollmentID UUID PRIMARY KEY,
    studentID VARCHAR(20) NOT NULL,
    courseID UUID NOT NULL,
    enrollmentDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    enrollmentStatus ENUM('ACTIVE', 'INACTIVE', 'WITHDRAWN',
'COMPLETED') NOT NULL,
    grade VARCHAR(5),
    attendancePercentage DECIMAL(5,2) DEFAULT 0.00,
    UNIQUE (studentID, courseID),
    FOREIGN KEY (studentID) REFERENCES Students(studentID) ON DELETE
CASCADE,
    FOREIGN KEY (courseID) REFERENCES Courses(courseID) ON DELETE
CASCADE
);

CREATE TABLE GeofenceLocations (
    geofenceID UUID PRIMARY KEY,
    centerLatitude DECIMAL(10,8) NOT NULL CHECK (centerLatitude
BETWEEN -90 AND 90),
    centerLongitude DECIMAL(11,8) NOT NULL CHECK (centerLongitude
BETWEEN -180 AND 180),
    radiusInMeters INTEGER NOT NULL CHECK (radiusInMeters BETWEEN 10
AND 1000),
    altitudeMin DECIMAL(8,2),
    altitudeMax DECIMAL(8,2),
    locationName VARCHAR(200) NOT NULL,
    building VARCHAR(100) NOT NULL,
    floor INTEGER CHECK (floor IS NULL OR floor BETWEEN -5 AND 50),
    roomNumber VARCHAR(20),
    capacity INTEGER,
    facilities JSON,
    accessRequirements JSON,
    isActive BOOLEAN NOT NULL DEFAULT TRUE,
    createdDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    lastCalibrated TIMESTAMP,
    accuracyLevel ENUM('HIGH', 'MEDIUM', 'LOW') NOT NULL DEFAULT
'MEDIUM'
);
CREATE TABLE ClassSchedules (
```

```
    scheduleID UUID PRIMARY KEY,
    courseID UUID NOT NULL,
    dayOfWeek INTEGER NOT NULL CHECK (dayOfWeek BETWEEN 1 AND 7),
    startTime TIME NOT NULL,
    endTime TIME NOT NULL,
    geofenceID UUID NOT NULL,
    isRecurring BOOLEAN NOT NULL DEFAULT TRUE,
    effectiveStartDate DATE NOT NULL,
    effectiveEndDate DATE,
    sessionType ENUM('LECTURE', 'LAB', 'TUTORIAL', 'EXAM', 'SEMINAR',
'ONLINE') NOT NULL,
    attendanceWindow INTEGER NOT NULL CHECK (attendanceWindow BETWEEN
5 AND 60),
    lateThreshold INTEGER NOT NULL CHECK (lateThreshold BETWEEN 1 AND
30),
    autoCloseAfter INTEGER NOT NULL CHECK (autoCloseAfter BETWEEN 60
AND 300),
    reminderBefore INTEGER NOT NULL CHECK (reminderBefore BETWEEN 5
AND 60),
    isActive BOOLEAN NOT NULL DEFAULT TRUE,
    createdBy VARCHAR(20) NOT NULL,
    createdDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    lastModified TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (courseID) REFERENCES Courses(courseID) ON DELETE
CASCADE,
    FOREIGN KEY (geofenceID) REFERENCES GeofenceLocations(geofenceID)
ON DELETE RESTRICT,
    FOREIGN KEY (createdBy) REFERENCES Lecturers(facultyID) ON DELETE
SET NULL
);




CREATE TABLE CourseSessions (
    sessionID UUID PRIMARY KEY,
```

```
    courseID UUID NOT NULL,
    sessionDate DATE NOT NULL,
    startTime TIME NOT NULL,
    endTime TIME NOT NULL,
    actualStartTime TIMESTAMP,
    actualEndTime TIMESTAMP,
    sessionType ENUM('LECTURE', 'LAB', 'TUTORIAL', 'SEMINAR', 'EXAM',
'ONLINE') NOT NULL,
    locationDescription VARCHAR(200) NOT NULL,
    sessionStatus ENUM('SCHEDULED', 'OPEN', 'CLOSED', 'CANCELLED',
'COMPLETED') NOT NULL,
    attendanceCount INTEGER DEFAULT 0,
    expectedAttendance INTEGER NOT NULL,
    sessionNotes TEXT,
    recordingURL VARCHAR(500),
    materials JSON,
    createdBy VARCHAR(20) NOT NULL,
    createdDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    modifiedDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    isRecurring BOOLEAN DEFAULT FALSE,
    recurringPattern JSON,
    FOREIGN KEY (courseID) REFERENCES Courses(courseID) ON DELETE
CASCADE,
    FOREIGN KEY (createdBy) REFERENCES Lecturers(facultyID) ON DELETE
SET NULL
);
```

4.3. Attendance and Verification Tables

```
CREATE TABLE FacialData (
    facialDataID UUID PRIMARY KEY,
```

```
    studentID VARCHAR(20) UNIQUE NOT NULL,
    facialEmbedding BLOB NOT NULL,
    encodingVersion VARCHAR(10) NOT NULL,
    algorithmUsed VARCHAR(50) NOT NULL,
    registrationDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    lastUpdated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updateReason VARCHAR(100),
    isActive BOOLEAN NOT NULL DEFAULT TRUE,
    confidenceThreshold DECIMAL(4,3) NOT NULL CHECK
(confidenceThreshold BETWEEN 0.500 AND 0.999),
    trainingImages INTEGER NOT NULL CHECK (trainingImages BETWEEN 1
AND 10),
    imageQualityScore DECIMAL(4,3) CHECK (imageQualityScore IS NULL OR
imageQualityScore BETWEEN 0.000 AND 1.000),
    lastSuccessfulMatch TIMESTAMP,
    failedMatchCount INTEGER DEFAULT 0,
    needsRetraining BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (studentID) REFERENCES Students(studentID) ON DELETE
CASCADE
);

CREATE TABLE AttendanceRecords (
    attendanceID UUID PRIMARY KEY,
    studentID VARCHAR(20) NOT NULL,
    sessionID UUID NOT NULL,
    courseID UUID NOT NULL,
    checkInTime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    checkOutTime TIMESTAMP,
    attendanceStatus ENUM('PRESENT', 'ABSENT', 'LATE', 'EXCUSED',
'PARTIAL') NOT NULL,
    arrivalStatus ENUM('ON_TIME', 'LATE', 'VERY_LATE'),
    lateMinutes INTEGER CHECK (lateMinutes IS NULL OR lateMinutes
BETWEEN 0 AND 120),
    verificationMethod ENUM('SELF_CHECKIN', 'LECTURER_ASSISTED',
'MANUAL_ENTRY') NOT NULL,
    locationLatitude DECIMAL(10,8) NOT NULL,
    locationLongitude DECIMAL(11,8) NOT NULL,
    locationAccuracy DECIMAL(8,2) NOT NULL,
    distanceFromGeofence DECIMAL(8,2) NOT NULL,
```

```
    faceMatchScore DECIMAL(4,3) CHECK (faceMatchScore IS NULL OR
faceMatchScore BETWEEN 0.000 AND 1.000),
    faceMatchAttempts INTEGER DEFAULT 1,
    deviceInfo JSON NOT NULL,
    ipAddress VARCHAR(45) NOT NULL,
    userAgent VARCHAR(500),
    batteryLevel INTEGER CHECK (batteryLevel IS NULL OR batteryLevel
BETWEEN 0 AND 100),
    networkType VARCHAR(20),
    isVerified BOOLEAN NOT NULL DEFAULT FALSE,
    verifiedBy VARCHAR(20),
    verificationDate TIMESTAMP,
    notes TEXT,
    photoURL VARCHAR(500),
    isDisputed BOOLEAN DEFAULT FALSE,
    lastModified TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (studentID) REFERENCES Students(studentID) ON DELETE
CASCADE,
    FOREIGN KEY (sessionID) REFERENCES CourseSessions(sessionID) ON
DELETE CASCADE,
    FOREIGN KEY (courseID) REFERENCES Courses(courseID) ON DELETE
CASCADE,
    FOREIGN KEY (verifiedBy) REFERENCES Users(userID) ON DELETE SET
NULL
);
```

4.4. Leave and Dispute Management Tables

```
CREATE TABLE LeaveRequests (
    leaveRequestID UUID PRIMARY KEY,
    studentID VARCHAR(20) NOT NULL,
    courseID UUID,
    sessionID UUID,
    leaveType ENUM('MEDICAL', 'PERSONAL', 'EMERGENCY', 'FAMILY',
'ACADEMIC', 'OTHER') NOT NULL,
    severity ENUM('LOW', 'NORMAL', 'HIGH', 'CRITICAL') NOT NULL
DEFAULT 'NORMAL',
    startDate DATE NOT NULL,
    endDate DATE NOT NULL,
    totalDays INTEGER,
```

```
    halfDay BOOLEAN DEFAULT FALSE,
    timeSlot ENUM('MORNING', 'AFTERNOON'),
    reason TEXT NOT NULL CHECK(LENGTH(reason) <= 1000),
    supportingDocument JSON,
    doctorNote BOOLEAN DEFAULT FALSE,
    emergencyContact VARCHAR(20),
    alternateContact VARCHAR(20),
    requestDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status ENUM('PENDING', 'APPROVED', 'DENIED', 'CANCELLED') NOT NULL
DEFAULT 'PENDING',
    urgency ENUM('LOW', 'NORMAL', 'HIGH', 'URGENT') NOT NULL DEFAULT
'NORMAL',
    autoApproval BOOLEAN DEFAULT FALSE,
    reviewedBy VARCHAR(20),
    reviewDate TIMESTAMP,
    reviewComments TEXT,
    approvalDate TIMESTAMP,
    notificationSent BOOLEAN DEFAULT FALSE,
    followUpRequired BOOLEAN DEFAULT FALSE,
    followUpDate DATE,
    FOREIGN KEY (studentID) REFERENCES Students(studentID) ON DELETE
CASCADE,
    FOREIGN KEY (courseID) REFERENCES Courses(courseID) ON DELETE SET
NULL,
    FOREIGN KEY (sessionID) REFERENCES CourseSessions(sessionID) ON
DELETE SET NULL,
    FOREIGN KEY (reviewedBy) REFERENCES Users(userID) ON DELETE SET
NULL,
    CHECK (endDate >= startDate)
);




CREATE TABLE Disputes (
    disputeID UUID PRIMARY KEY,
    studentID VARCHAR(20) NOT NULL,
```

```
    sessionID UUID,
    courseID UUID NOT NULL,
    attendanceRecordID UUID,
    disputeType ENUM('ATTENDANCE_ERROR', 'TECHNICAL_ISSUE',
'GEOFENCE_ERROR', 'FACE_RECOGNITION_FAIL', 'SYSTEM_MALFUNCTION',
'OTHER', 'LEAVE_RELATED') NOT NULL,
    disputeCategory ENUM('TECHNICAL', 'PROCEDURAL', 'ADMINISTRATIVE',
'APPEAL') NOT NULL,
    severity ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL') NOT NULL
DEFAULT 'MEDIUM',
    disputeReason VARCHAR(500) NOT NULL,
    description TEXT NOT NULL CHECK (LENGTH(description) <= 2000),
    expectedResolution TEXT,
    evidenceFile JSON,
    witnessInfo TEXT,
    timeOfIncident TIMESTAMP,
    deviceUsed JSON,
    locationDuringIncident JSON,
    submissionDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status ENUM('PENDING', 'UNDER_REVIEW', 'RESOLVED', 'REJECTED',
'ESCALATED') NOT NULL DEFAULT 'PENDING',
    priority ENUM('LOW', 'MEDIUM', 'HIGH', 'URGENT') NOT NULL DEFAULT
'MEDIUM',
    escalationLevel INTEGER DEFAULT 0 CHECK (escalationLevel BETWEEN 0
AND 3),
    assignedTo VARCHAR(20),
    assignmentDate TIMESTAMP,
    estimatedResolutionDate DATE,
    actualResolutionDate TIMESTAMP,
    resolutionTime INTEGER,
    resolutionComments TEXT,
    finalAction ENUM('ATTENDANCE_CORRECTED', 'REQUEST_DENIED',
'TECHNICAL_FIX', 'POLICY_EXCEPTION', 'NO_ACTION'),
    studentSatisfaction INTEGER CHECK (studentSatisfaction IS NULL OR
studentSatisfaction BETWEEN 1 AND 5),
    followUpRequired BOOLEAN DEFAULT FALSE,
    relatedDisputes JSON,
    FOREIGN KEY (studentID) REFERENCES Students(studentID) ON DELETE
CASCADE,
```

```
    FOREIGN KEY (sessionID) REFERENCES CourseSessions(sessionID) ON
DELETE SET NULL,
    FOREIGN KEY (courseID) REFERENCES Courses(courseID) ON DELETE
CASCADE,
    FOREIGN KEY (attendanceRecordID) REFERENCES
AttendanceRecords(attendanceID) ON DELETE SET NULL,
    FOREIGN KEY (assignedTo) REFERENCES Users(userID) ON DELETE SET
NULL
);
```

4.5. System Support and Administration Tables

```
CREATE TABLE Notifications (
    notificationID UUID PRIMARY KEY,
    recipientID VARCHAR(20) NOT NULL,
    senderID VARCHAR(20),
    notificationType ENUM('SESSION_ALERT', 'DISPUTE_UPDATE',
'LEAVE_STATUS', 'SYSTEM_ALERT', 'REMINDER', 'ANNOUNCEMENT', 'GENERAL')
NOT NULL,
    category ENUM('ATTENDANCE', 'ACADEMIC', 'SYSTEM', 'PERSONAL',
'EMERGENCY') NOT NULL,
    channel ENUM('PUSH', 'EMAIL', 'SMS', 'IN_APP') NOT NULL,
    title VARCHAR(200) NOT NULL,
    message TEXT NOT NULL CHECK (LENGTH(message) <= 1000),
    richContent JSON,
    priority ENUM('LOW', 'MEDIUM', 'HIGH', 'URGENT') NOT NULL DEFAULT
'MEDIUM',
    isRead BOOLEAN NOT NULL DEFAULT FALSE,
    readDate TIMESTAMP,
    scheduledTime TIMESTAMP,
    sentTime TIMESTAMP,
    deliveryStatus ENUM('PENDING', 'SENT', 'DELIVERED', 'FAILED') NOT
NULL DEFAULT 'PENDING',
    deliveryAttempt INTEGER DEFAULT 0,
    expiryTime TIMESTAMP,
    actionURL VARCHAR(500),
    metadata JSON,
    groupID VARCHAR(50),
    batchID VARCHAR(50),
    deviceToken VARCHAR(255),
```

```
    createdDate TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    lastModified TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (recipientID) REFERENCES Users(userID) ON DELETE
CASCADE,
    FOREIGN KEY (senderID) REFERENCES Users(userID) ON DELETE SET NULL
);
```

4.6. Data Types and Constraints

The schema utilizes appropriate SQL data types (UUID, VARCHAR, TEXT, INTEGER, DECIMAL, BOOLEAN, TIMESTAMP, DATE, TIME, ENUM, JSON, BLOB) as per the "Data Elements" document. Key constraints include:

- **PRIMARY KEY:** Uniquely identifies each row in a table.
- **FOREIGN KEY:** Enforces referential integrity between tables. ON DELETE clauses define behavior on deletion of referenced records.
- **UNIQUE:** Ensures values in a column or set of columns are unique.
- **NOT NULL:** Ensures a column cannot have a NULL value.
- **DEFAULT:** Specifies a default value if none is provided during insertion.
- **CHECK:** Enforces specific conditions on data values (e.g., ranges, formats).

4.7. Normalization

The schema aims for Third Normal Form (3NF) to reduce data redundancy and improve data integrity. This is achieved by ensuring atomic values, full dependency on primary keys, and minimizing transitive dependencies. Associative tables like CourseEnrollments resolve many-to-many relationships.

4.8. Impact of NoSQL on Design Logic

Because Firestore is a NoSQL system:

- **Redundancy is acceptable** for performance (e.g., course name may be duplicated in session records).
- **Denormalization** allows fast access with fewer reads, which is important for mobile apps.
- **Collections are flat**, not nested—subcollections are only used when necessary.
- **Relationships are maintained through ID references**, not enforced with foreign key constraints like in SQL.

In addition to indexes automatically created for PRIMARY KEY and UNIQUE constraints, the following performance-enhancing secondary indexes are defined:

```
CREATE INDEX idx_attendance_student_date ON
AttendanceRecords(studentID, checkInTime);
CREATE INDEX idx_attendance_session ON AttendanceRecords(sessionID);
CREATE INDEX idx_attendance_course_date ON AttendanceRecords(courseID,
checkInTime);
CREATE INDEX idx_sessions_course_date ON CourseSessions(courseID,
sessionDate);
CREATE INDEX idx_enrollments_student ON CourseEnrollments(studentID);
CREATE INDEX idx_enrollments_course ON CourseEnrollments(courseID);
CREATE INDEX idx_notifications_recipient_read ON
Notifications(recipientID, isRead);
CREATE INDEX idx_disputes_status_submission ON Disputes(status,
submissionDate);
CREATE INDEX idx_disputes_student ON Disputes(studentID);
CREATE INDEX idx_disputes_assigned_to ON Disputes(assignedTo);
CREATE INDEX idx_audit_logs_user_date ON AuditLogs(userID, timestamp);
CREATE INDEX idx_audit_logs_resource ON AuditLogs(resourceType,
resourceID);
CREATE INDEX idx_facial_data_student_active ON FacialData(studentID,
isActive);
CREATE INDEX idx_leave_requests_student_status ON
LeaveRequests(studentID, status);
CREATE INDEX idx_leave_requests_reviewed_by ON
LeaveRequests(reviewedBy);
CREATE INDEX idx_device_registrations_user ON
DeviceRegistrations(userID);
```

This design philosophy makes the system **scalable, fast**, and **real-time capable**, with the trade-off being a need for thoughtful indexing and validation.

5. Database Implementation

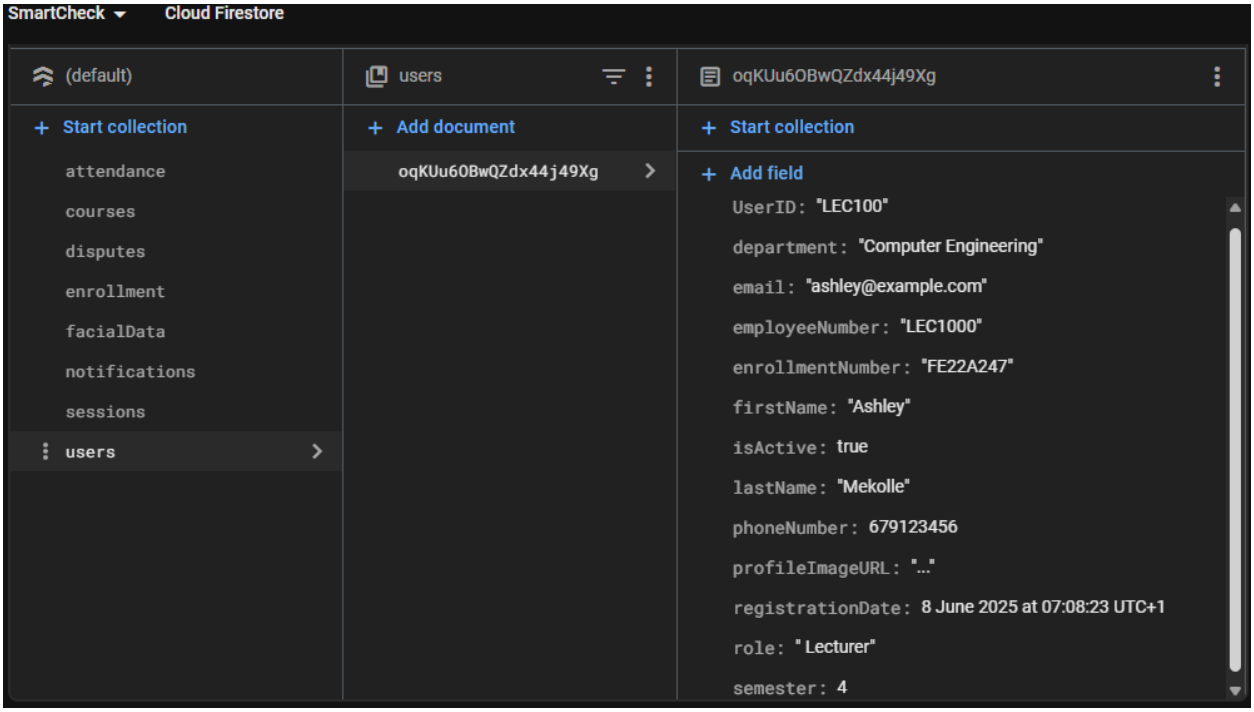
This section explains how the conceptual model is realized in Firebase Firestore. Each major entity from the conceptual design becomes a Firestore collection, and fields are designed to optimize read performance and security.

5.1. Core Collections and Document Structure

Below is an overview of how the key entities are implemented in Firestore:

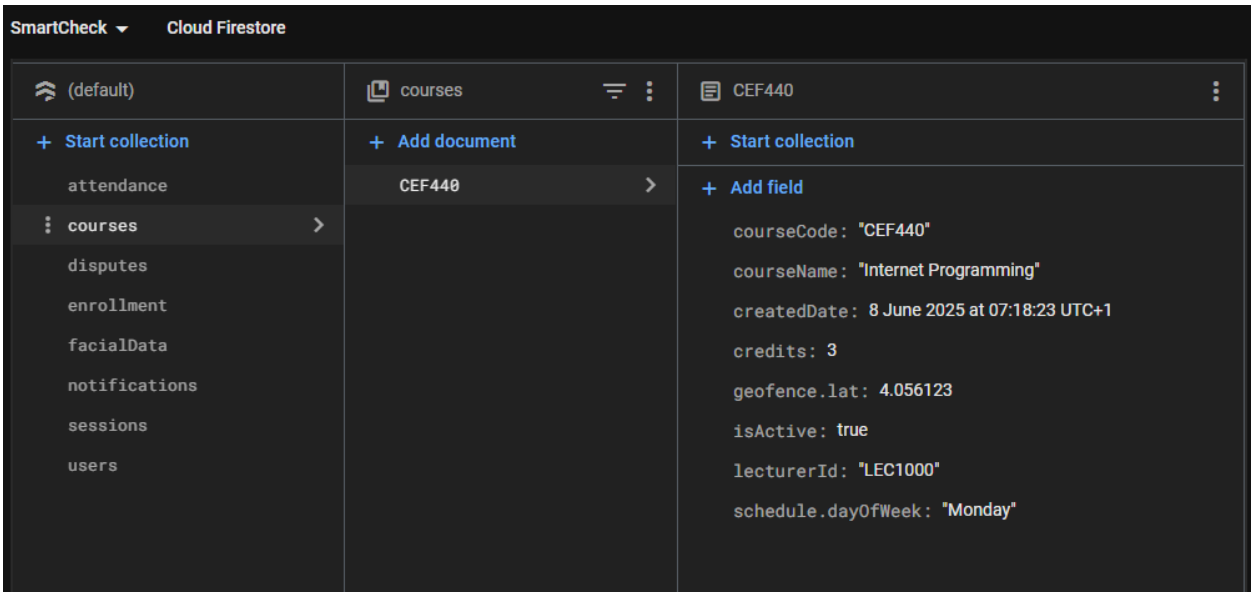
5.1.1. Users collection

Stores each user's metadata (name, email, role, department, etc.), plus profile image URL and facial data reference. Users are distinguished by a role field (e.g., student, lecturer, admin).



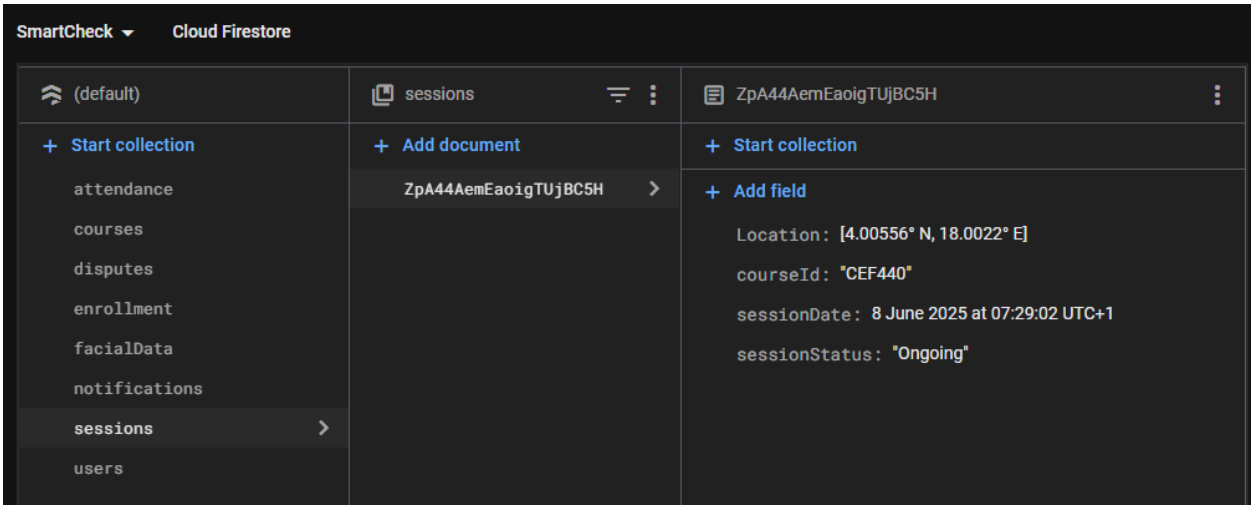
5.1.2. courses collection

Contains documents for each course. Fields include course code, title, lecturer ID, enrolled students (as an array of user IDs or references), and optional tags like semester or department.



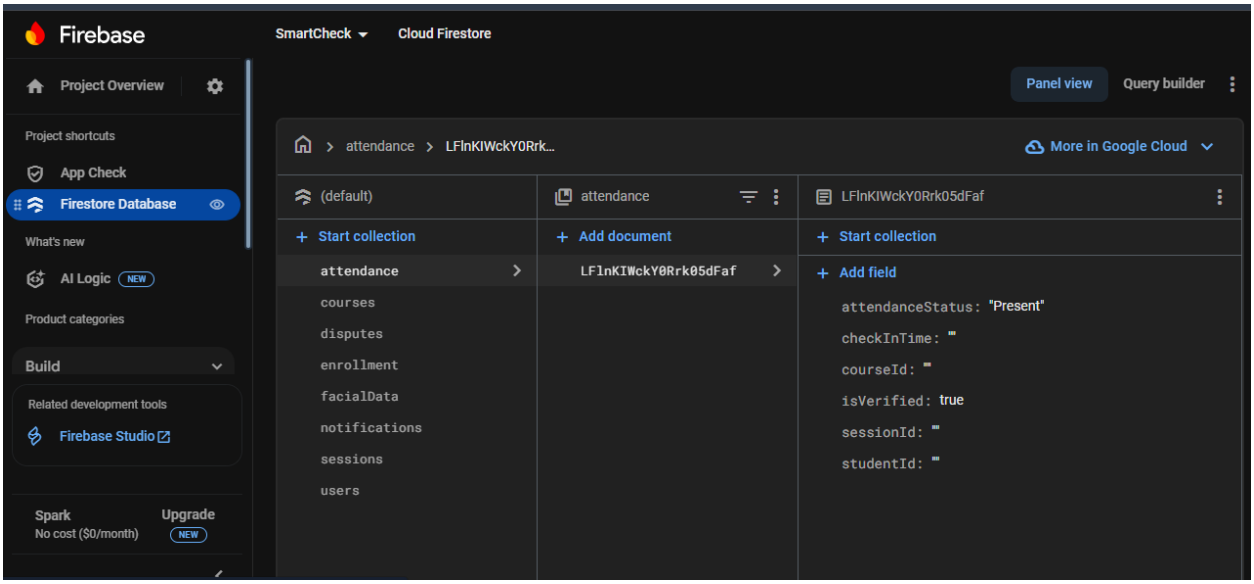
5.1.3. sessions collection

Each document represents a single attendance session. Fields include course ID, lecturer ID, geofence coordinates, radius, start/end time, and status (open, closed). Sessions may also store summary fields (like total check-ins) to improve performance.



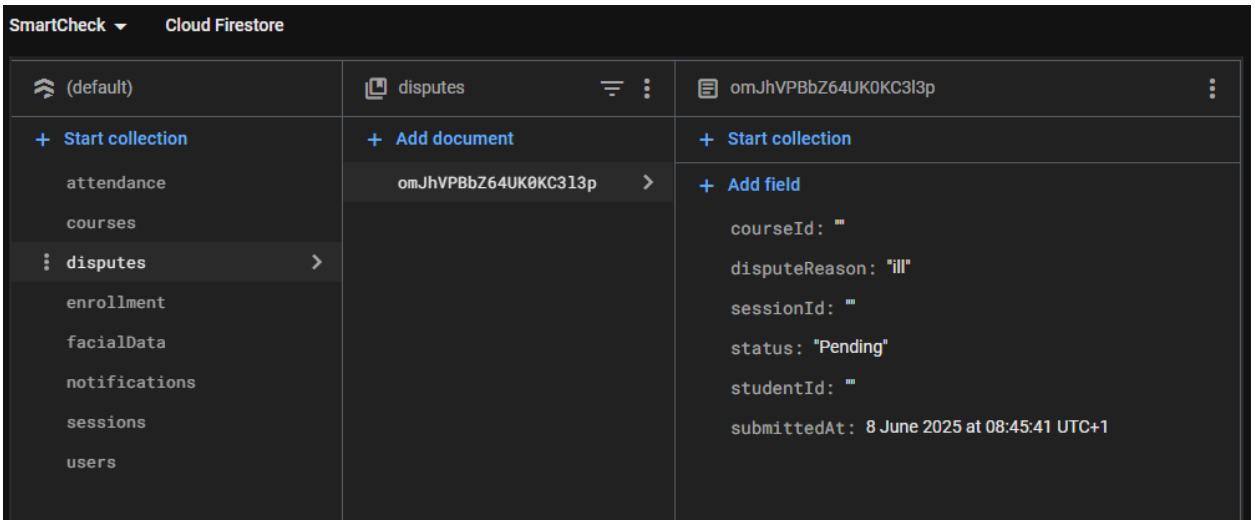
5.1.4. attendance collection

Logs individual check-ins. Each record includes the student's ID, session ID, timestamp, GPS location, face match score, and result status (e.g., Present, Late, Rejected). Attendance records are immutable after creation, except when manually edited by lecturers.



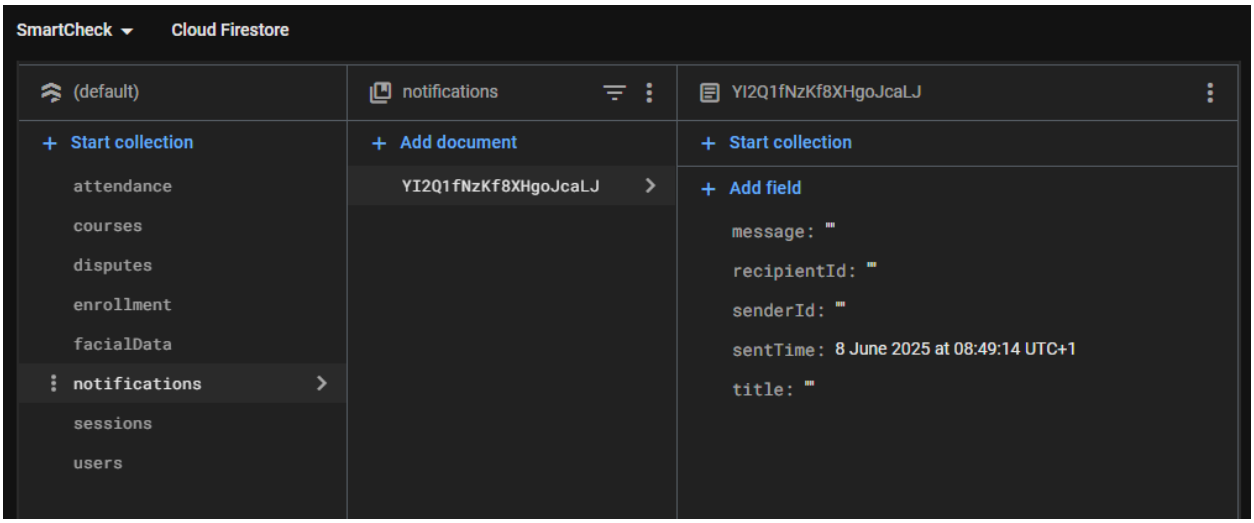
5.1.5. disputes collection

Stores student-raised issues or leave requests. Each document contains the session ID, student ID, reason, optional proof document (URL), status (pending, approved, denied), and timestamps.



5.1.6. notifications collection

Stores FCM tokens and user/device associations for push notifications.



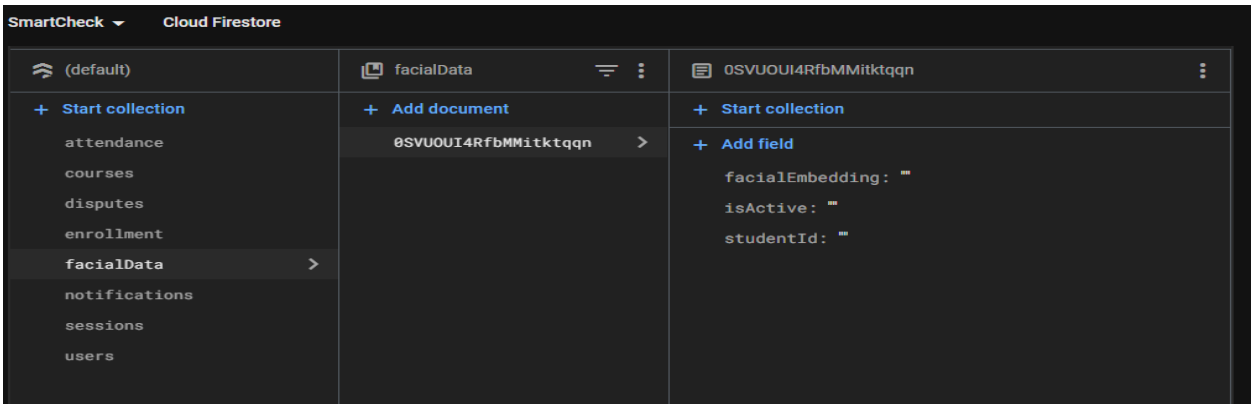
Each collection is implemented as a **top-level collection**, with document IDs generated automatically or using custom formats (e.g., course codes).

5.2. Storing Geolocation and Facial Recognition Data

- **Geolocation:**
  Each session document includes fields like latitude, longitude, and radius to define the geofence. During check-in, the system compares the device's coordinates with these to determine validity.

- **Facial Recognition:**
  Facial reference data may be stored as:

  - A **URL** to a secure Firebase Storage image (if image-based recognition is used), or
  - A **face embedding vector** (numerical features extracted from face) stored as a string or array in the users collection.
    Each attendance record stores the **face match confidence score** from the recognition process.



5.3. Document Validation and Schema Integrity

Firestore is schema-less by nature, but developers can enforce consistency by:

- Using **server-side validation** in the backend (e.g., via Express controllers).
- Implementing **Firebase Security Rules** to restrict what fields can be written by whom.
- Optionally using tools like **Firestore schema validators** or **TypeScript interfaces** in the backend logic.

5.4. Indexing Strategies

To support high-performance queries:

- Firestore automatically indexes most fields.
- Compound indexes are manually created to support filters (e.g., query sessions by course and status).
- Attendance history queries are indexed by userId and timestamp to support pagination in student dashboards.

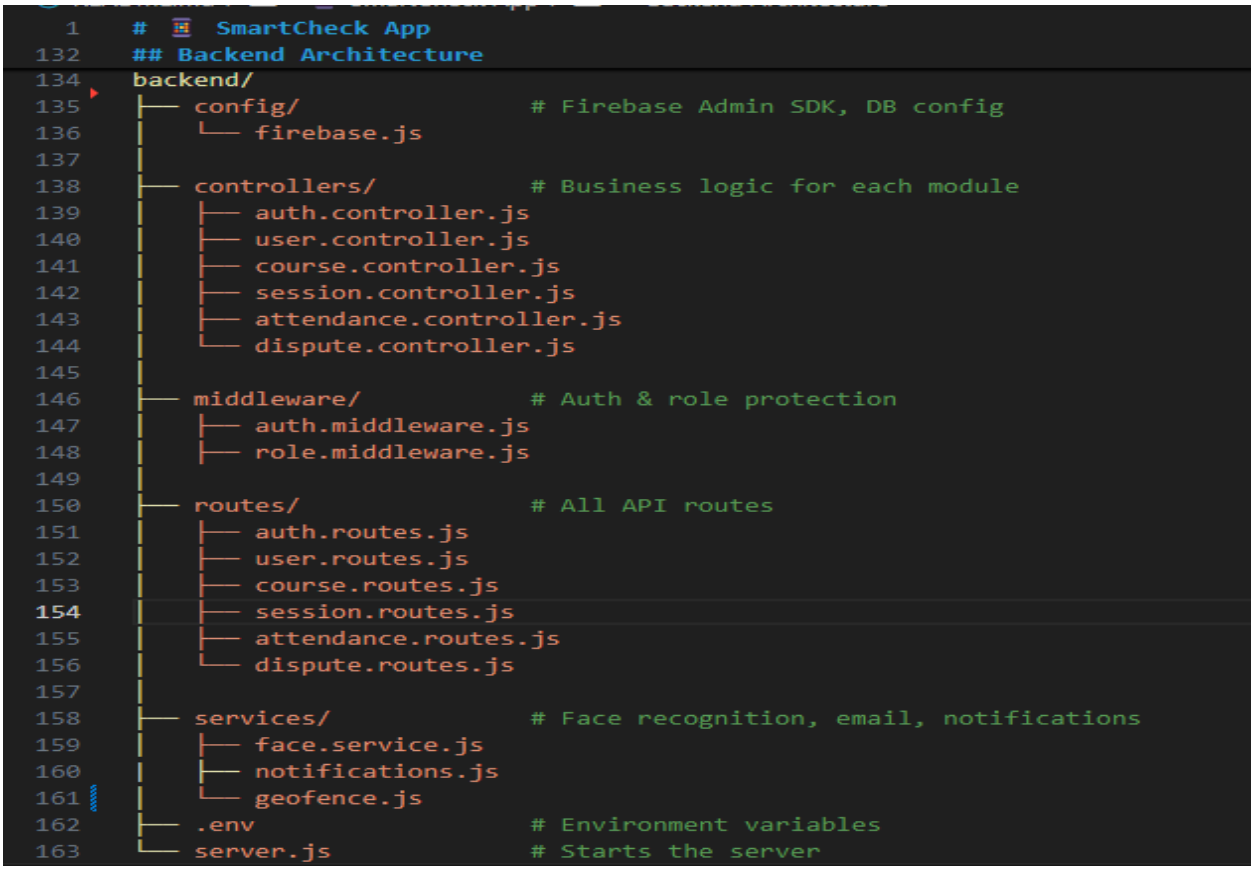
5.5. Security Rules (Firestore Specific)

Security rules are written to:

- Allow users to read/write only their own data.
- Allow lecturers to modify only their course and session records.
- Restrict sensitive collections like attendance and disputes from being edited after finalization (unless by admin).
- Enforce role-based permissions using Firebase Authentication claims.

6. Backend Implementation

The SmartCheck backend is built with **Node.js + Express** and leverages the **Firebase Admin SDK** to perform all database operations securely. The Express application acts as a middle layer between the React Native frontend and Firestore, exposing RESTful endpoints that map directly to service-level functions.



6.1. Interaction Pattern

Each incoming request passes through **authentication middleware**, which verifies the Firebase ID-token and enriches req.user with role claims (student, lecturer, admin). Controller methods then delegate to **service functions** that encapsulate Firestore logic, ensuring a clean separation of concerns:

6.2. Service-Function Structure

- **UserService** – createUser, getUserById, updateUser, deleteUser
- **CourseService** – createCourse, enrollStudent, listCoursesByUser
- **SessionService** – startSession, closeSession, getSessionsByCourse
- **AttendanceService** – logAttendance, batchAssistedCheckIn, getHistoryByUser

- **DisputeService** – raiseDispute, resolveDispute, listDisputes

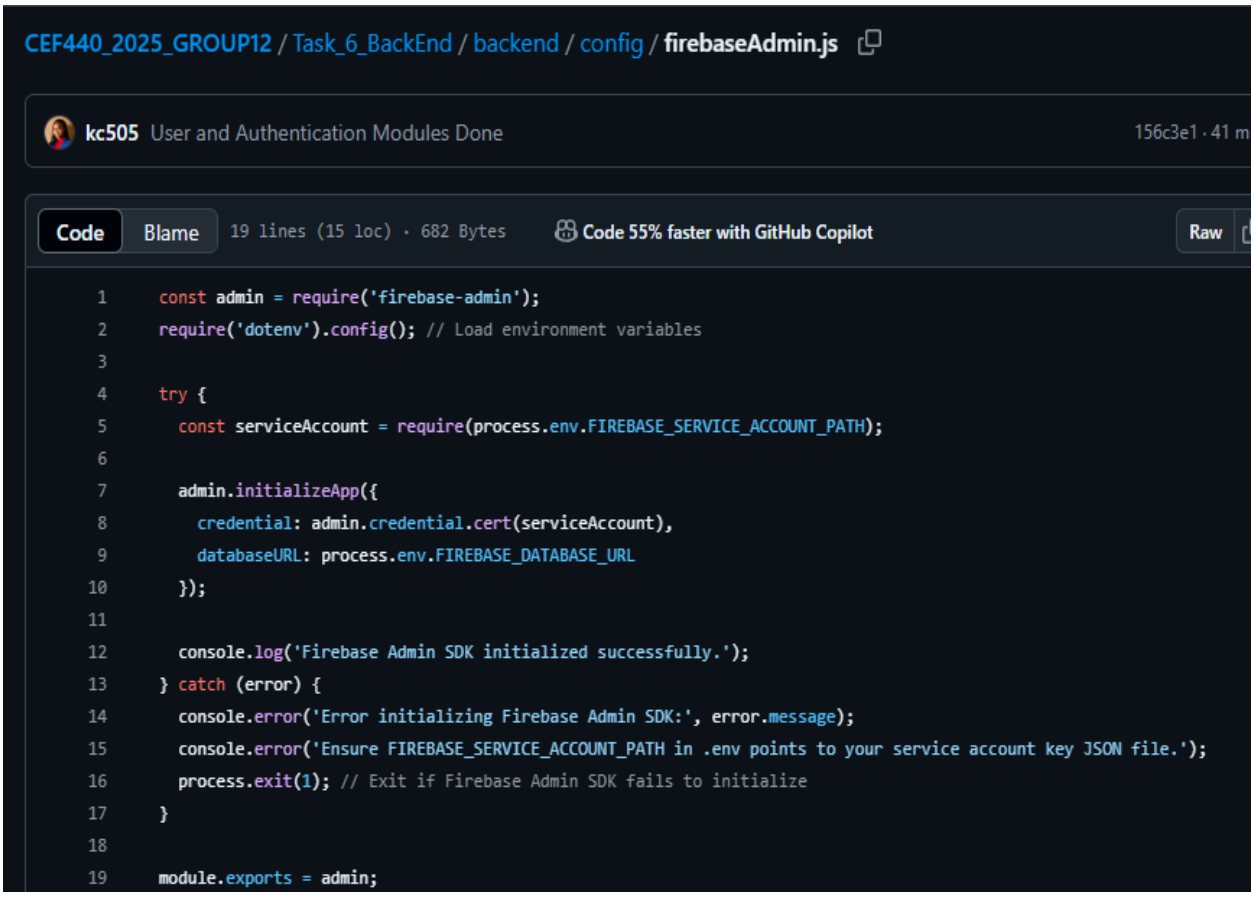
Each service function performs:

1. **Input validation** (checking required fields, data types, latitude/longitude bounds).
2. **Role enforcement** (e.g., only lecturers can startSession).
3. **Firestore transaction or batched write** to guarantee atomicity for multi-document updates (e.g., logging attendance and incrementing session counters in the same commit).

7. Connecting Database to Backend

7.1. Firebase Setup

The backend initializes Firebase by importing a **service-account key** and calling admin.initializeApp. The key file path and other environment-specific variables (e.g., storage bucket) are **referenced through environment variables** loaded by dotenv, keeping secrets out of source control.



8. Conclusion

The database design and implementation for the SmartCheck system provides a strong, scalable, and secure foundation for managing attendance in a mobile-first environment. By leveraging Firebase Firestore, the system ensures real-time data synchronization, efficient storage of biometric and geolocation data, and seamless integration with authentication and cloud services.

Through a carefully structured conceptual model and clearly defined data entities—such as users, courses, sessions, and attendance logs—SmartCheck achieves both operational efficiency and data integrity. The backend, powered by Node.js and the Firebase Admin SDK, adds essential layers of logic, validation, and role-based access control, ensuring that all interactions with the database are secure, consistent, and traceable.

Together, these components enable SmartCheck to deliver a modern attendance solution that is intuitive for users, robust for administrators, and adaptable for future growth.